



UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática



Prof. André Kishimoto

Computação Visual

Projeto de Computação Visual

Diogo Cornelio Martins Rosa – 10403852
José Victor Scheurich Roling– 10418225
Milton Almeida Leoncio– 10416764
Vitor Neudl Gandolfi – 10408845

São Paulo, 2026



Análise Inicial

1. Para que o projeto seja desenvolvido usando as versões mais recentes e estáveis das bibliotecas, o que mudará em relação aos vídeos/repositório da disciplina?

Diferente dos vídeos que utilizam SDL3 3.4.0 e SDL3_Image 3.4.0, foram usadas as bibliotecas SDL3 3.4.16 e SDL3_Image 3.4.6. Não existe grande variação entre elas, apresentam apenas algumas correções de bugs e melhorias de compatibilidade com consoles e outras plataformas.

2. O processo de build usado pelo grupo é o mesmo dos vídeos/repositório da disciplina? Se for diferente, descreva brevemente o processo adotado pelo grupo.

O processo de build é o mesmo apresentado no vídeo e repositório. As bibliotecas foram propriamente linkadas no ambiente do VS Code, foram passados os parâmetros apropriados no arquivo tasks.json e a compilação é feita com o makefile disponibilizado sempre copiando a DLL automaticamente.

3. O grupo usará mais de um sistema operacional, compilador e/ou editor diferente? Por exemplo, Windows e Linux, Windows e Mac, GCC e Clang, GCC e MSVC, vscode e Visual Studio, XCode e vscode, vim e Notepad++, etc.

Será utilizado o sistema Windows, GCC e Visual Studio Code.



UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática



Prof. André Kishimoto
Computação Visual

Análise do escopo e funcionalidades

Em geral, os itens 1, 2, 3 e 4 podem ser em grande parte resolvidos a partir dos exemplos já apresentados. Os exemplos 5, 6, 7 e 8 possuem alguma relação com os materiais de estudo, mas ainda precisarão de alterações e será necessário pesquisar mais sobre determinados tópicos.

Detalhamento da análise de cada item:

1. Carregamento da imagem:

Os vídeos e códigos exemplificam o carregamento de imagens (*IMG_LoadTexture()*), mas será necessário criar uma função de validação do tipo de imagem uma vez que os exemplos apenas verificam se a imagem existe e foi carregada corretamente.

Os exemplos mostram o uso de logs para tratamento de erros (*SDL_Log()* e *SDL_GetError()*).

O modo de entrada da imagem também deverá ser modificada uma vez que o código base utiliza o nome do arquivo como uma constante, enquanto o projeto exige que a imagem seja recebida como argumento na linha de comando.

Principais referências:

- Códigos e vídeos sobre image e invert_image apresentados nos repositórios.

SDL:

- *IMG_LoadTexture()*.
- *SDL_Log()* e *SDL_GetError()*.
- Structs *SDL_Window* e *SDL_Renderer*.



Adicionar/Modificar:

- Adicionar entrada da imagem como argumento na linha de comando.

2. Análise e conversão para escala de cinza:

O uso de structs (*MyWindow* e *MyImage*) pode facilitar a manipulação das imagens e janelas.

O código *invert_image* apresenta uma função(*invert_image()*) que pode ser adaptada para a conversão dos pixels para escala de cinza, pois já realiza a leitura e alteração individual dos canais RGB. Entretanto, será necessário implementar uma função adicional para verificar se a imagem já está em escala de cinza e aplicar a fórmula de conversão especificada pelo projeto.

Principais referências:

- Códigos *filter_image* e *invert_image* apresentados nos repositórios.
- Função *invert_image()*.

SDL:

- Structs *MyWindow* e *MyImage*.
- *SDL_LockSurface()*, *SDL_UnlockSurface()* para iniciar e terminar a manipulação dos pixels da imagem.
- *SDL_GetPixelFormatDetails()*, *SDL_GetRGBA()* e *SDL_MapRGBA()* para controle e modificação de cada pixel.

Adicionar/Modificar:

- Adicionar função para verificar se a imagem está em escala de cinza.
- Adaptar a função *invert_image()* para converter a imagem para uma escala de cinza seguindo os parâmetros $Y = 0.2125 * r + 0.7154 * g + 0.0721 * b$.



UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática



Prof. André Kishimoto
Computação Visual

3. Interface gráfica de usuário (GUI) com duas janelas:

Os códigos fornecem conhecimento suficiente para criação, destruição, redimensionamento e posicionamento de apenas uma janela (`SDL_SetWindowSize()` e `SDL_SetWindowPosition()`). Sendo assim, seria necessário estudar e adaptar a estrutura para trabalhar com duas janelas simultaneamente. Como comentado no vídeo, o uso de structs pode facilitar a manipulação de janelas diferentes.

Principais referências:

- Códigos `filter_image` e `invert_image` apresentados nos repositórios.
- Fluxo de criação e adaptação de janela na função `main` em `invert_image()`.

SDL:

- Structs `MyWindow` e `MyImage`.
- `SDL_SetWindowSize()` e `SDL_SetWindowPosition()`.
- `SDL_GetWindowBordersSize()` é necessário para posicionar a janela secundária na posição (0,0) de forma adequada.

Adicionar/Modificar:

- Adicionar novas variáveis referentes à segunda janela com o uso das structs.
- Adaptar a função `Initialize()` para iniciar duas janelas.



4. Análise e exibição do histograma

Os exemplos de primitivas apresentam recursos que podem ser utilizados para desenhar o histograma na interface, como pontos, linhas e retângulos (*SDL_RenderLine*, *SDL_RenderPoint*, *SDL_RenderRect*). Entretanto, os exemplos não implementam o cálculo de um histograma de uma imagem. Será necessário estudar mais como calcular a frequência das intensidades dos pixels e como utilizar esses valores para gerar uma representação gráfica. Também será necessário definir, a partir da média e do desvio padrão, os critérios utilizados para classificar a imagem como clara, média ou escura e seu contraste como alto, médio ou baixo.

Principais referências:

- Código e vídeo sobre primitivas apresentado no repositório.
- Fluxo de criação e modificação de primitivos dentro da função loop().

SDL:

- Structs *SDL_Color*, *SDL_FRect* e *SDL_FPoint*.
- *SDL_SetRenderDrawColor()*, *SDL_RenderClear()*, *SDL_RenderPoints()*, *SDL_RenderPoint()*, *SDL_RenderRect()*, *SDL_RenderFillRect()* e *SDL_RenderPresent()*.
- Biblioteca *SDL_ttf*.

Adicionar/Modificar:

- Adicionar nova função para gerar um histograma correspondente à imagem.
- Adicionar nova função para avaliar o contraste e intensidade da imagem.



5. Equalização do histograma

As funções e exemplos apresentados no requisito 4 já apresentam uma base para criação de botões com primitivas e detecção de interações a partir do sistema de eventos do SDL.

Principais referências:

- Código e vídeo sobre primitivas apresentado no repositório.

SDL:

- Structs `SDL_Color`, `SDL_FRect` e `SDL_FPoint`.
- `SDL_SetRenderDrawColor()`, `SDL_RenderClear()`, `SDL_RenderPoints()`, `SDL_RenderPoint()`, `SDL_RenderRect()`, `SDL_RenderFillRect()` e `SDL_RenderPresent()`.

Adicionar/Modificar:

- Adaptar o sistema de eventos na função `loop()` para reconhecer interações de clique.
- Adaptar a função de histograma do requisito 4 para dispor botões em diferentes estados.

6. Exibição da imagem

Os requisitos relacionados às janelas podem ser implementados com um estudo melhor das funções apresentadas nos requisitos 2 e 3. Já o uso de texto não tem uma referência detalhada nos vídeos e repositórios. Sendo assim, será preciso avaliar possíveis funções ou bibliotecas adicionais para implementação e manipulação de texto no projeto como, por exemplo, o uso da biblioteca recomendada `SDL_ttf`.



UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática



Prof. André Kishimoto
Computação Visual

7. Salvar imagem

A documentação do SDL apresenta algumas alternativas para o salvamento de imagens como a função *SDL_SaveFile()*. Ainda assim, será necessário criar uma função que salva a imagem toda vez que, dentro do sistema de eventos, a tecla S for pressionada.

Principais referências:

- Código e vídeo sobre *invert_image* apresentado no repositório.

SDL:

- Função *SDL_SaveFile()*.

Adicionar/Modificar:

- Adicionar função para salvar a imagem. A mesma função também verifica se já existe um arquivo salvo e, caso exista, sobrescreve o arquivo. Caso contrário, a função cria um arquivo de imagem *output_image.png*.
- Adaptar o sistema de eventos na função *loop()* para reconhecer a tecla “S”.

8. Exibição de textos (informações no histograma e botões)

Assim como mencionado no requisito 6, o uso de fontes e manipulação de texto deve ser mais bem estudado para o desenvolvimento do projeto.